

Python Programming

Unit 02 – Lecture 04 Notes

Functions Basics and Parameters

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	2
2	Why Functions?	2
2.1	Functions in Real-world Applications	3
3	Function Syntax and Behavior	3
3.1	Basic Definition	3
3.2	Anatomy of a Function (Mental Model)	3
3.3	<code>return</code> vs <code>print</code>	4
3.4	Docstrings	4
4	Parameters and Arguments	4
4.1	Parameters vs Arguments	4
4.2	Positional and Keyword Arguments	4
5	Default Arguments	5
5.1	Pitfall: Mutable Default Arguments	5
5.2	Safe Pattern for Defaults	5
6	Variable-Length Arguments	5
6.1	<code>*args</code> (Extra Positional Arguments)	5
6.2	<code>**kwargs</code> (Extra Keyword Arguments)	5
6.3	Worked Example: Average of Any Number of Scores	6
7	Scope: LEGB (Local/Enclosed/Global/Built-in)	6
7.1	LEGB Scope Rule	6
7.2	Enclosed Scope and <code>nonlocal</code>	6
7.3	Global Variables (Use Carefully)	6
8	Passing Data to Functions: Mutable vs Immutable	7
9	Demo Walkthrough: Grade Calculator Function	7
9.1	What the Demo Is Teaching	7
9.2	Suggested Run Steps	7

10 Quiz and Solutions	7
10.1 Checkpoint 1 (Mutable Defaults)	7
10.2 Checkpoint 2 (Local Scope)	8
10.3 Checkpoint 3 (Viewing Docstrings)	8
10.4 Think-Pair-Share (Keyword Arguments)	8
10.5 Exit Question (Signature for Average)	8
11 Practice Exercises (With Solutions)	8
11.1 Exercise 1: Safe Append Helper	8
11.2 Exercise 2: Named Formatting Function	9
11.3 Exercise 3: Return vs Print	9
12 Further Reading	9

1 Lecture Overview

Functions let you name a block of logic and reuse it. This is one of the most important habits for writing clean programs: instead of copying code, you call a function.

This lecture covers:

- function definition syntax (`def`, parameters, `return`),
- docstrings and why documentation matters,
- positional vs keyword arguments,
- default arguments (and the common mutable-default trap),
- variable-length arguments (`*args`, `**kwargs`), and
- local vs global scope.

2 Why Functions?

Functions improve code quality in multiple ways:

- **Avoid repetition:** write logic once, reuse many times.
- **Readability:** a good name summarizes the intent.
- **Testing:** small functions are easier to test than long scripts.
- **Modularity:** break a big program into small, manageable parts.
- **Maintainability:** change logic in one place instead of many copies.
- **Abstraction:** callers use the function without knowing internal details.

2.1 Functions in Real-world Applications

Functions are everywhere in real systems:

- **ATM software:** `withdraw()`, `deposit()`, `check_balance()`.
- **Data processing:** clean values, transform formats, compute summaries.
- **Web development:** request handlers and reusable helpers (validation, formatting).
- **Games:** update score/state, compute collisions, apply rules.

The practical benefit is not “shorter code” — it is code that is easier to read, test, and change safely.

3 Function Syntax and Behavior

3.1 Basic Definition

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)
```

Key parts:

- `def` starts a function definition.
- Parameters (`a`, `b`) are inputs.
- `return` sends a value back to the caller.

3.2 Anatomy of a Function (Mental Model)

Every function definition has the same core parts:

- `def` keyword
- function name (use clear verbs: `compute_avg`, `validate_email`)
- parameter list (inputs)
- indented body (statements)
- optional `return` (output)

When you *call* a function, Python:

1. evaluates the arguments,
2. assigns them to parameters,
3. executes the body,
4. returns the value (or `None` if there is no return).

3.3 return vs print

A common beginner confusion is mixing output with return values.

- **return** gives a value back to the caller for further use.
- **print** only displays text; it does not produce a reusable value.

3.4 Docstrings

A docstring is the first string in a function body. It documents the purpose, inputs, and output.

```
def area(r):  
    """Return the area of a circle with radius r."""  
    return 3.14 * r * r
```

Docstrings can be viewed with `help(area)` or via `area.__doc__`. **Docstring tips (good habits):**

- Start with a one-line summary in an imperative style (“Return ...”, “Compute ...”).
- Mention parameters and return value clearly.
- If the function has edge cases, document them (e.g., “returns 0 for empty input”).

4 Parameters and Arguments

4.1 Parameters vs Arguments

- **Parameters** appear in the function definition.
- **Arguments** are the actual values passed during a call.

Example:

```
def greet(name, msg): # parameters  
    return f"{msg}, {name}"  
  
print(greet("Asha", "Hello")) # arguments
```

4.2 Positional and Keyword Arguments

```
def power(base, exp):  
    return base ** exp  
  
power(2, 3) # positional  
power(base=2, exp=3) # keyword  
power(exp=3, base=2) # keyword order can change
```

Keyword arguments improve readability when a function has many parameters.

5 Default Arguments

Defaults make a function flexible:

```
def greet(name, msg="Hello"):
    return f"{msg}, {name}"
```

Important detail: default values are evaluated once, when the function is defined (not every time it is called).

5.1 Pitfall: Mutable Default Arguments

This is one of the most common Python interview and exam traps.

```
def add_item(x, items=[]):
    items.append(x)
    return items

print(add_item(1)) # [1]
print(add_item(2)) # [1, 2] (surprise!)
```

Why it happens: the default list `[]` is created once and reused.

5.2 Safe Pattern for Defaults

Use `None` and create a new list inside the function:

```
def add_item_safe(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
```

6 Variable-Length Arguments

Sometimes you do not know how many inputs the caller will provide.

6.1 `*args` (Extra Positional Arguments)

```
def total(*nums):
    return sum(nums)

print(total(10, 20, 30))
```

Inside the function, `nums` is a tuple of all extra positional arguments.

6.2 `**kwargs` (Extra Keyword Arguments)

```
def describe(**info):
    return info

print(describe(name="Asha", age=18))
```

Inside the function, `info` is a dictionary of keyword arguments.

6.3 Worked Example: Average of Any Number of Scores

```
def avg(*scores):
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```

The empty-input check avoids division by zero.

7 Scope: LEGB (Local/Enclosed/Global/Built-in)

7.1 LEGB Scope Rule

Python resolves names using the **LEGB** rule:

- **Local:** inside the current function
- **Enclosed:** inside an outer (enclosing) function
- **Global:** module-level variables
- **Built-in:** Python built-ins like `len`, `sum`

By default, variables created inside a function are **local**.

```
x = 5

def foo():
    x = 2 # local
    return x

print(foo(), x)
```

7.2 Enclosed Scope and nonlocal

When you have a function inside a function, the inner function can *read* outer variables. If you want to *assign* to an enclosed variable, use **nonlocal**.

```
def make_counter():
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc
```

7.3 Global Variables (Use Carefully)

Using global variables is possible (with the **global** keyword), but it makes code harder to reason about and test. Prefer returning values instead.

8 Passing Data to Functions: Mutable vs Immutable

Python uses “object references” when passing arguments. Practically:

- **Immutable types** (int, float, str, tuple) cannot be changed in place. Rebinding a parameter creates a new object.
- **Mutable types** (list, dict, set) can be modified in place, so changes inside the function may be visible to the caller.

Example:

```
def bump(x):  
    x += 1 # new int, caller does not change  
  
def add_item(lst, item):  
    lst.append(item) # mutates list, caller sees the change
```

The same idea applies to dictionaries:

```
def set_grade(record, grade):  
    record["grade"] = grade # mutates the dict
```

If you want “no side effects” behavior, copy first (e.g., `lst.copy()`).

9 Demo Walkthrough: Grade Calculator Function

Script: `demo/grade_calculator.py`

9.1 What the Demo Is Teaching

- Use a function to keep grading logic in one place.
- Use parameters/defaults so the grading scheme can be reused.
- Return a grade value (instead of only printing).

9.2 Suggested Run Steps

```
python demo/grade_calculator.py
```

Extension idea: allow custom grade boundaries (e.g., a different university policy).

10 Quiz and Solutions

10.1 Checkpoint 1 (Mutable Defaults)

Question. What happens if you use a mutable default like `def f(items=[])`?

Solution. The default object is created once and reused across calls. So changes to `items` in one call affect the default in later calls.

10.2 Checkpoint 2 (Local Scope)

Question. Predict the output:

```
x = 5

def foo():
    x = 2
    return x

print(foo(), x)
```

Solution. Output is 2 5. The x inside foo shadows the global x but does not change it.

10.3 Checkpoint 3 (Viewing Docstrings)

Question. How can you view a function's docstring while programming? Give one method.

Solution. Two common ways are:

- `help(func)` in the Python REPL/notebook, or
- `print(func.__doc__)` to print the docstring string.

10.4 Think-Pair-Share (Keyword Arguments)

Prompt. When do keyword arguments improve readability?

Sample response. When a function has multiple parameters of the same type or meaning, keywords prevent confusion. For example, `resize(width=640, height=480)` is clearer than `resize(640, 480)`.

10.5 Exit Question (Signature for Average)

Question. Write a function signature that accepts any number of scores and returns their average.

Solution.

```
def avg(*scores):
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```

11 Practice Exercises (With Solutions)

11.1 Exercise 1: Safe Append Helper

Task. Write a function `push(x, items=None)` that appends x to a list, but does not use a mutable default.

Solution.

```
def push(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
```


11.2 Exercise 2: Named Formatting Function

Task. Write `format_name(first, last, title="")` that returns a formatted full name. Prefer keyword arguments for optional parts.

Solution.

```
def format_name(first, last, title=""):
    title = title.strip()
    if title:
        return f"{title} {first} {last}"
    return f"{first} {last}"
```

11.3 Exercise 3: Return vs Print

Task. Write a function `is_even(n)` that returns `True` or `False`. Do not print inside the function.

Solution.

```
def is_even(n):
    return n % 2 == 0
```

12 Further Reading

- <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Define reusable functions with parameters and return values
- Use positional, keyword, default, and variable-length arguments
- Write and access docstrings for documentation
- Explain LEGB scope (local/enclosed/global/built-in)
- Predict behavior when passing mutable vs immutable data

Why Functions?

- Avoid repetition
- Improve readability
- Make code easier to test
- Enable modular design

Functions in Real-world Applications

- ATM software: `withdraw()`, `deposit()`, `check_balance()`
- Data processing: clean, transform, and summarize datasets
- Web development: route handlers and reusable utilities
- Game design: update player state, collision checks, scoring

Function Syntax

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)
```

- `def` creates a function
- `return` sends a value back

Anatomy of a Function

- `def` keyword
- Function name (use meaningful verbs: `compute_avg`)
- Parameters (inputs)
- Function body (indented block)
- Optional `return` statement (output)

Docstrings

- Triple-quoted string inside a function
- Describe purpose, parameters, and return value
- Access with `help(func)` or `func.__doc__`

```
def area(r):  
    """Return area of a circle."""  
    return 3.14 * r * r
```

Parameters vs Arguments

- Parameters appear in function definition
- Arguments are values passed in a call
- Positional and keyword arguments both valid

Default Arguments

- Provide a fallback value
- Evaluated at function definition time
- Avoid mutable defaults (like lists)

```
def greet(name, msg="Hello"):  
    return f"{msg}, {name}"
```

Pitfall: Mutable Default Arguments

- Default objects are created once and reused
- A list default is shared across calls

```
def add_item(x, items=[]):  
    items.append(x)  
    return items  
  
print(add_item(1)) # [1]  
print(add_item(2)) # [1, 2] (surprise!)
```

Variable-Length Arguments

- `*args` collects extra positional arguments
- `**kwargs` collects extra keyword arguments

```
def total(*nums):  
    return sum(nums)
```

Worked Example: Average of Any Number of Scores

```
def avg(*scores):  
    if not scores:  
        return 0.0  
    return sum(scores) / len(scores)  
  
print(avg(10, 20, 30))
```

- Handle empty input to avoid division by zero

Scope: LEGB (Local/Enclosed/Global/Built-in)

- Python uses LEGB name resolution (Local, Enclosed, Global, Built-in)
- Variables inside functions are local by default
- Prefer returning values instead of modifying globals

Passing Data: Mutable vs Immutable

- Immutable types (int, float, str, tuple) cannot be changed in place
- Mutable types (list, dict, set) can be modified inside a function
- Document side effects or copy inputs for “no-surprise” behavior

```
def bump(x):  
    x += 1 # creates a new int  
  
def add_item(lst, item):  
    lst.append(item) # mutates the list
```

Demo: Grade Calculator Function

- Compute average score
- Return letter grade based on thresholds
- Use default parameters for flexibility
- Extension: allow custom boundaries for other grading schemes

Script: demo/grade_calculator.py

Checkpoint 1

What happens if you use a mutable default like `def f(items=[])`?

Checkpoint 2

Predict the output:

```
x = 5  
  
def foo():  
    x = 2  
    return x  
  
print(foo(), x)
```

Checkpoint 3

How can you view a function's docstring while programming? Give one method.

Think-Pair-Share

When do keyword arguments improve readability? Give a short example.

Summary

- Functions organize and reuse logic
- Arguments can be positional, keyword, or variable-length
- Docstrings document intent and usage
- Scope rules help avoid bugs

Exit Question

Write a function signature that accepts any number of scores and returns their average.